where $\langle \boldsymbol{A}, \boldsymbol{B} \rangle_F = \sum_{i,j} A_{i,j} B_{i,j}$ denotes the Frobenius inner product between real-valued matrices $\boldsymbol{A}$ and $\boldsymbol{B}$. Conveniently, eq. (1) constitutes a "linear assignment problem" (LAP) (Bertsekas, 1998) for which efficient, practical algorithms are known. Having solved this assignment problem on each layer, we can then permute the weights of model B to match model A as closely as possible

$$\boldsymbol{W}'_\ell = \boldsymbol{P}_\ell \boldsymbol{W}_\ell^{(B)} \boldsymbol{P}_{\ell-1}^\top, \quad \boldsymbol{b}'_\ell = \boldsymbol{P}_\ell \boldsymbol{b}_\ell^{(B)}$$

for each layer ℓ, producing weights Θ' with activations that align as closely possible with Θ_A.

Computationally, this entire process is relatively lightweight: the $\boldsymbol{Z}^{(A)}$ and $\boldsymbol{Z}^{(B)}$ matrices can be computed in a single pass over the training dataset, and, in practice, a full run through the training dataset may be unnecessary. Solving eq. (1) is possible due to well-established, polynomial-time algorithms for solving the linear assignment problem (Kuhn, 2010; Jonker & Volgenant, 1987; Crouse, 2016). Also, conveniently, the activation matching at each layer is independent of the matching at every other layer, resulting in a separable and straightforward optimization problem; this advantage will not be enjoyed by the following methods.

Dispensing with regression, one could similarly associate units by matching against a matrix of cross-correlation coefficients in place of $\boldsymbol{Z}^{(A)}(\boldsymbol{Z}^{(B)})^\top$. We observed correlation matching to work equally well but found OLS regression matching to be more principled and easier to implement.

Activation matching has previously been studied for model merging in Tatro et al. (2020); Singh & Jaggi (2020); Li et al. (2016) albeit not from the perspective of OLS regression.

3.2 Matching Weights

Instead of associating units by their activations, we could alternatively inspect the weights of the model itself. Consider the first layer weights, $\boldsymbol{W}_1$; each row of $\boldsymbol{W}_1$ corresponds to a single feature. If two such rows were equal, they would compute exactly the same feature (ignoring bias terms for the time being). And, if $[\boldsymbol{W}_1^{(A)}]_{i,:} \approx [\boldsymbol{W}_1^{(B)}]_{j,:}$, it stands to reason that units i and j should be associated. Extending this idea to every layer, we are inspired to pursue the optimization

$$\arg\min_\pi \ \|\text{vec}(\Theta_A) - \text{vec}(\pi(\Theta_B))\|^2 \ = \ \arg\max_\pi \ \text{vec}(\Theta_A) \cdot \text{vec}(\pi(\Theta_B)).$$

We can re-express this in terms of the full weights,

$$\arg\max_{\pi=\{\boldsymbol{P}_i\}} \langle \boldsymbol{W}_1^{(A)}, \boldsymbol{P}_1 \boldsymbol{W}_1^{(B)} \rangle_F + \langle \boldsymbol{W}_2^{(A)}, \boldsymbol{P}_2 \boldsymbol{W}_2^{(B)} \boldsymbol{P}_1^\top \rangle_F + \cdots + \langle \boldsymbol{W}_L^{(A)}, \boldsymbol{W}_L^{(B)} \boldsymbol{P}_{L-1}^\top \rangle_F, \quad (2)$$

resulting in another matching problem. We term this formulation the "sum of bilinear assignments problem" (SOBLAP). Unfortunately, this matching problem is thornier than the classic linear assignment matching problem presented in eq. (1). Unlike LAP, we are interested in permuting *both* the rows and columns of $\boldsymbol{W}_\ell^{(B)}$ to match $\boldsymbol{W}_\ell^{(A)}$, which fundamentally differs from permuting only rows or only columns. We formalize this difficulty as follows.

Lemma 1. *The sum of a bilinear assignments problem (SOBLAP) is NP-hard and admits no polynomial-time constant-factor approximation scheme for $L > 2$.*

Lemma 1 contrasts starkly with classical LAP, for which polynomial-time algorithms are known.

Undeterred, we propose a approximation algorithm for SOBLAP. Looking at a single $\boldsymbol{P}_\ell$ while holding the others fixed, we observe that the problem can be reduced to a classic LAP,

$$\begin{aligned} &\arg\max_{\boldsymbol{P}_\ell} \langle \boldsymbol{W}_\ell^{(A)}, \boldsymbol{P}_\ell \boldsymbol{W}_\ell^{(B)} \boldsymbol{P}_{\ell-1}^\top \rangle_F + \langle \boldsymbol{W}_{\ell+1}^{(A)}, \boldsymbol{P}_{\ell+1} \boldsymbol{W}_{\ell+1}^{(B)} \boldsymbol{P}_\ell^\top \rangle_F \\ &\qquad = \arg\max_{\boldsymbol{P}_\ell} \langle \boldsymbol{P}_\ell, \boldsymbol{W}_\ell^{(A)} \boldsymbol{P}_{\ell-1} (\boldsymbol{W}_\ell^{(B)})^\top + (\boldsymbol{W}_{\ell+1}^{(A)})^\top \boldsymbol{P}_{\ell+1} \boldsymbol{W}_{\ell+1}^{(B)} \rangle_F. \end{aligned}$$

This leads to a convenient coordinate descent algorithm: go through each layer and greedily select its best $\boldsymbol{P}_\ell$. Repeat until convergence. We present this in Algorithm 1.

Although we present Algorithm 1 in terms of an MLP without bias terms, in practice our implementation can handle the weights of models of nearly arbitrary architectures, including bias terms, residual connections, convolutional layers, attention mechanisms, and so forth. We propose an extension of Algorithm 1 to merging more than two models at a time in Appendix A.10.

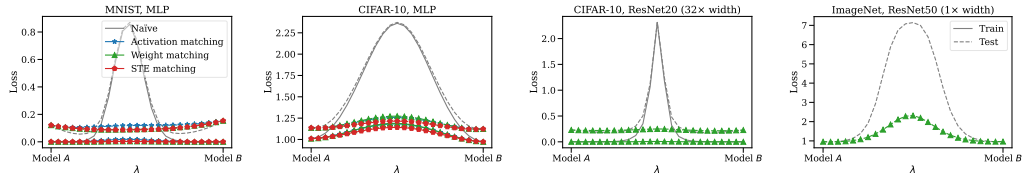

Figure 2: **Linear mode connectivity is possible after permuting.** Loss landscapes when interpolating between models trained on MNIST, CIFAR-10, and ImageNet. In all cases we can significantly improve over naïve interpolation. Straight-through estimator matching performs best but is very computationally expensive. Weight and activation matching perform similarly, although weight matching is orders of magnitude faster and does not rely on the input data distribution. We hypothesize that the ImageNet barrier could be reduced by increasing the model width as in Section 5.3.

Algorithm 1: PERMUTATIONCOORDINATEDESCENT

Given: Model weights $\Theta_A = \left\{\boldsymbol{W}_1^{(A)}, \ldots, \boldsymbol{W}_L^{(A)}\right\}$ and $\Theta_B = \left\{\boldsymbol{W}_1^{(B)}, \ldots, \boldsymbol{W}_L^{(B)}\right\}$

Result: A permutation $\pi = \{\boldsymbol{P}_1, \ldots, \boldsymbol{P}_{L-1}\}$ of Θ_B such that $\text{vec}(\Theta_A) \cdot \text{vec}(\pi(\Theta_B))$ is approximately maximized.

Initialize: $\boldsymbol{P}_1 \leftarrow \boldsymbol{I}, \ldots, \boldsymbol{P}_{L-1} \leftarrow \boldsymbol{I}$
repeat
  for $\ell \in$ RANDOMPERMUTATION$(1, \ldots, L-1)$ **do**
    $\boldsymbol{P}_\ell \leftarrow$ SOLVELAP $\left(\boldsymbol{W}_\ell^{(A)} \boldsymbol{P}_{\ell-1} (\boldsymbol{W}_\ell^{(B)})^\top + (\boldsymbol{W}_{\ell+1}^{(A)})^\top \boldsymbol{P}_{\ell+1} \boldsymbol{W}_{\ell+1}^{(B)}\right)$
  end
until *convergence*

Lemma 2. *Algorithm 1 terminates.*

Our experiments showed this algorithm to be fast in terms of both iterations necessary for convergence and wall-clock time, generally on the order of seconds to a few minutes.

Unlike the activation matching method presented in Section 3.1, weight matching ignores the data distribution entirely. Ignoring the input data distribution and therefore the loss landscape handicaps weight matching but allows it to be much faster. We therefore anticipate its potential application in fields such as finetuning (Devlin et al., 2019; Wortsman et al., 2022b;a), federated learning (McMahan et al., 2017; Konečný et al., 2016a;b), and model patching (Matena & Raffel, 2021; Sung et al., 2021; Raffel, 2021). In practice, we found weight matching to be surprisingly competitive with data-aware methods. Section 5 studies this trade-off.

3.3 LEARNING PERMUTATIONS WITH A STRAIGHT-THROUGH ESTIMATOR

Inspired by the success of straight-through estimators (STEs) in other discrete optimization problems (Bengio et al., 2013; Kusupati et al., 2021; Rastegari et al., 2016; Courbariaux & Bengio, 2016), we attempt here to "learn" the ideal permutation of weights $\pi(\Theta_B)$. Specifically, our goal is to optimize

$$\min_{\tilde{\Theta}_B} \mathcal{L}\left(\frac{1}{2}\left(\Theta_A + \text{proj}\left(\tilde{\Theta}_B\right)\right)\right), \qquad \text{proj}(\Theta) \triangleq \arg\max_{\pi} \text{vec}(\Theta) \cdot \text{vec}(\pi(\Theta_B)), \tag{3}$$

where $\tilde{\Theta}_B$ denotes an approximation of $\pi(\Theta_B)$, allowing us to implicitly optimize π. However, eq. (3) involves inconvenient non-differentiable projection operations, $\text{proj}(\cdot)$, complicating the optimization. We overcome this via a "straight-through" estimator: we parameterize the problem in terms of a set of weights $\tilde{\Theta}_B \approx \pi(\Theta_B)$. In the forward pass, we project $\tilde{\Theta}_B$ to the closest realizable $\pi(\Theta_B)$. In the backwards pass, we then switch back to the unrestricted weights $\tilde{\Theta}_B$. In this way, we

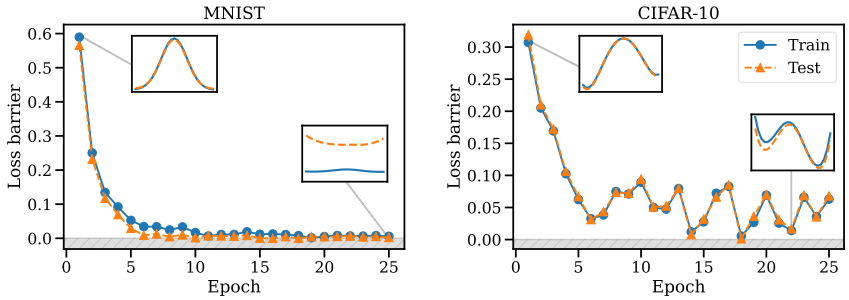

Figure 3: **Linear mode connectivity is challenging at initialization.** We show loss barriers per training time for MLPs trained on MNIST (left) and CIFAR-10 (right). Loss interpolation plots are inlaid to highlight results in initial and later epochs. LMC manifests gradually throughout training. We hypothesize that the variance in CIFAR-10 training is higher due to our MLP architecture being under-powered relative to the dataset. (Y-axis scales differ in each inlaid plot.)

are guaranteed to stay true to the projection constraints in evaluating the loss but can still compute usable gradients at our current parameters, $\tilde{\Theta}_B$.[2]

Conveniently, we can re-purpose Algorithm 1 to solve $\text{proj}(\tilde{\Theta}_B)$. Furthermore, we found that initializing $\tilde{\Theta}_B = \Theta_A$ performed better than random initialization. This is to be expected immediately at initialization since the initial matching will be equivalent to the weight matching method of Section 3.1. However, it is not immediately clear why these solutions continue to outperform a random initialization asymptotically.

Unlike the aforementioned methods, Algorithm 2 attempts to explicitly "learn" the best permutation π using a conventional training loop. By initializing to the weight matching solution of Section 3.2 and leveraging the data distribution as in Section 3.1, it seeks to offer a best-of-both-worlds solution. However, this comes at a very steep computational cost relative to the other two methods.

4 A Counterexample to Universal Linear Mode Connectivity

In this section we argue that common optimization algorithms, especially SGD and its relatives, are implicitly biased towards solutions admitting linear mode connectivity. In particular, we demonstrate – by way of a counterexample – that adversarial, non-SGD solutions exist in loss landscapes such that no permutation of units results in linear mode connectivity. We present this counterexample in complete detail in Appendix A.6.

The existence of adversarial basins suggests that our ability to find LMC between independently trained models is thanks to inherent biases in optimization methods. We emphasize that this counterexample does not contradict Conjecture 1; rather, it illustrates the importance of the conjecture's restriction to SGD solutions (Entezari et al., 2021). Characterizing the precise mechanism by which these solutions are biased towards LMC could be an exciting avenue for future work.

We also note that there are invariances beyond permutation symmetries: It is possible to move features between layers, re-scale layers, and so forth. Prior works noted the feature/layer association (Nguyen et al., 2021) and re-scaling invariances (Ainsworth et al., 2018). The importance of these other symmetries and their interplay with optimization algorithms remains unclear.

5 Experiments

Our base methodology is to separately train two models, A and B, starting from different random initializations and with different random batch orders, resulting in trained weights Θ_A and Θ_B,

[2]Note again that projecting according to inner product distance is equivalent to projecting according to the L_2 distance when parameterizing the estimator based on the B endpoint. We also experimented with learning the midpoint directly, $\tilde{\Theta} \approx \frac{1}{2}(\Theta_A + \pi(\Theta_B))$, in which case the L_2 and inner product projections diverge. In testing all possible variations, we found that optimizing the B endpoint had a slight advantage, but all possible variations performed admirably.